

TCOP: Two-Timescale Service Caching Coordination and Pairwise Q-Learning Offloading for Heterogeneous Edge Systems

Liang Chen*

*School of Computer Science
South China Normal University
Guangzhou 510631, Guangdong, China
2024023503@m.scnu.edu.cn*

Abstract—Service caching and task offloading jointly affect service-loading, inter-task transfer, and computation delays for directed acyclic graph (DAG) applications with service-dependent tasks in edge systems with heterogeneous cache capacities and computing capabilities. Independent local-demand caching limits coordination of network-wide service coverage, while offloading must consider task requirements, cached services, and server states. We propose TCOP, a two-timescale joint optimization method. At the slow timescale, a central coordinator runs Demand-Driven Coordinated Caching (DCC) to configure next-period caches from historical requests and existing service placement under per-server capacity constraints. At the fast timescale, each entry server uses a Pairwise Dueling Double Deep Q-Network (Pairwise PD3QN) to select the current task’s execution server. In frozen-policy testing in the main environment, TCOP achieved a mean application completion time of 0.2668 s, 65.51% lower than DAOC and 11.09% lower than discrete SAC with the same DCC caching mechanism. Module comparisons and cache metrics show that DCC expands service coverage and Pairwise PD3QN further improves cache utilization, reducing the fraction of requests requiring remote service loading. Simulations evaluate five scientific workflow types and joint scaling of server count and total cache slots.

Index Terms—edge computing, task offloading, service caching, resource scheduling, deep reinforcement learning

I. INTRODUCTION

Mobile edge computing brings computing and services closer to resource-constrained devices [1]. Many applications comprise service-dependent tasks organized as a directed acyclic graph (DAG); their completion time depends on computation, service loading, and inter-task data transfer [2], [3], [4], [5].

As shown in Fig. 1, an entry server assigns ready tasks to cooperating servers with limited, heterogeneous cache capacities and computing capabilities [6], [7]. Cache misses require remote service loading, while dependencies may require data transfers between execution servers.

Independent-task offloading studies address energy optimization under queuing-delay constraints [8] and adaptation to changing delay–energy preferences and server configurations [9]. DAG studies combine dependency-aware offloading with resource allocation [10], workflow partitioning and pricing [11], or edge–cloud scheduling [12], without jointly determining cross-server service caching.

Joint approaches use hierarchical learning [7], delay-constrained on-demand caching and dependent-task offloading [13], caching before batch-workflow scheduling [14], multi-timescale caching, offloading, and resource allocation for sequentially dependent tasks [15], or service criticality and demand [16]. DAOC periodically updates caches from local demand and loading costs, selects servers with a local deep Q-network (DQN), and exchanges cache states [5].

Frequent joint network-wide caching and offloading updates increase online decision overhead. Independent local-request caching [5] cannot readily account for network-wide coverage and remote loading costs; excessive duplication may limit coverage. Conventional flat DQNs map servers to fixed action-value output positions. Double-Q and Dueling improve value estimation without changing this mapping [17]–[19].

TCOP combines demand-driven coordinated caching (DCC) with a pairwise dueling double deep Q-network (Pairwise PD3QN) at different timescales. Its main contributions are as follows.

(1) We establish a joint optimization model that accounts for DAG dependencies, service loading, and heterogeneous cache capacities. With mean application completion time as the objective, the model incorporates periodic service cache configuration and per-task offloading into a single problem. We adopt centralized coordination at the slow timescale and entry-server offloading decisions at the fast timescale.

(2) We design DCC to coordinate service caching across servers. Subject to per-server capacity constraints, DCC combines historical demand, service loading-delay savings, server execution feedback, and the existing replica distribution to score candidate caching locations. It then determines the cache configuration for the next period through coverage repair and update acceptance rules.

(3) We design the Pairwise PD3QN offloading policy. For the current task, the policy estimates the action value of each candidate server using a shared server encoding and scoring structure. It learns offloading decisions from rewards that reflect increments in application completion time, and experiments with different module combinations evaluate the roles of caching coordination and the offloading policy.

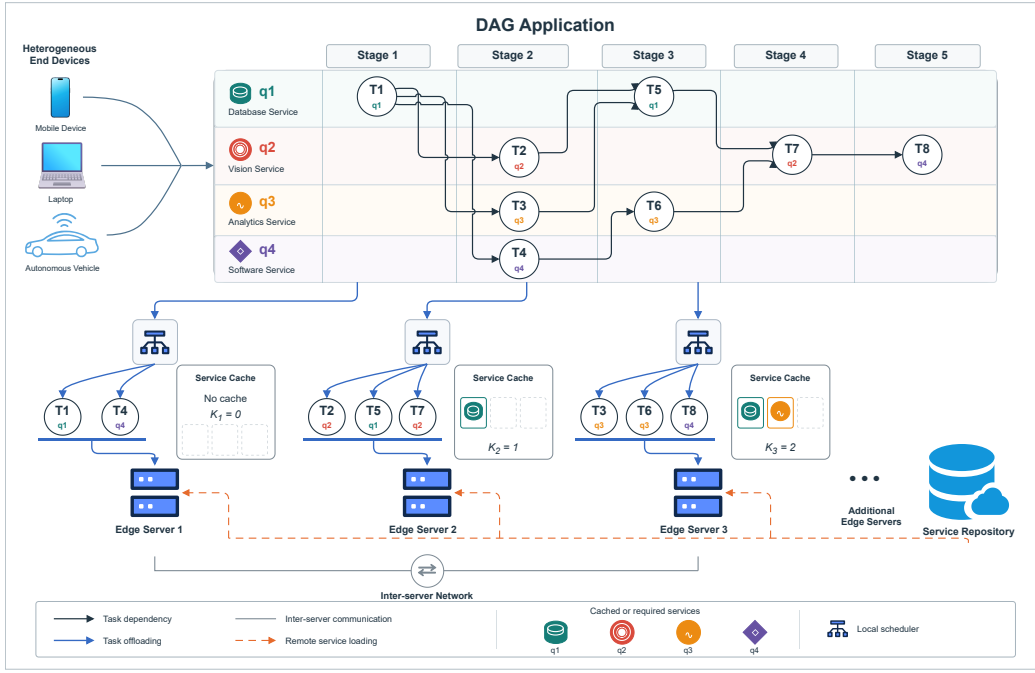


Fig. 1. DAG service caching and task offloading in a heterogeneous edge environment.

Experiments on five types of Pegasus workflows [2], [20] evaluate overall performance, mechanisms, module combinations, and sensitivity to workflow type and joint server/cache-slot scaling.

II. SYSTEM MODEL AND PROBLEM FORMULATION

A. System and Task Model

Mobile users submit service-dependent DAG applications to the nearest edge server (the entry server), which selects execution locations among reachable servers. Edge servers exchange task data and service images; the cloud supplies images only and does not execute tasks [3]–[5].

Let $m = 1, \dots, M$, $s = 1, \dots, S$, and $q = 1, \dots, Q$ index application instances, servers, and services. Server s has CPU frequency f_s and cache capacity K_s , with one slot per service. Capacities $K = (K_1, \dots, K_S)$ may differ, totaling B slots: $K_s \in \{0, 1, \dots, Q\}$, $s = 1, \dots, S$. $B = \sum_{s=1}^S K_s$.

Cache configuration $X^{(k)} = [x_{s,q}^{(k)}]$ is set at the start of period k and fixed throughout it; $x_{s,q}^{(k)}$ is 1 if server s caches service q , and 0 otherwise. Per-server capacity limits are imposed in P1.

The task dependencies of application m are represented as $G_m = (V_m, E_m)$, with $E_m \subseteq V_m \times V_m$.

Here, V_m contains real tasks and E_m their dependencies: edge (j, i) requires task j to finish and its output to arrive before i executes. Task i 's immediate predecessors and successors are $\text{Pred}_{m,i}$ and $\text{Succ}_{m,i}$; $d_{m,j,i}$ is the data size transferred from j to i [2]. The exit set is $V_m^{\text{out}} = \{i \in V_m : \text{Succ}_{m,i} = \emptyset\}$.

The complete DAG and task attributes are known at admission. The environment supplies tasks in a predefined de-

TABLE I
MAIN NOTATION AND DEFINITIONS

Symbol	Definition
$c_{m,i}$	Task CPU cycles
$d_{m,i}^{\text{in}}$	Task initial input size
$d_{m,j,i}$	Data size from predecessor to current task
$q_{m,i}$	Required service type
f_s	Server CPU frequency
W_s	Exogenous server background workload
K_s	Server cache capacity
B	Total cache slots
L_q	Service image size
$\ell_{s',s}^{(k)}$	Inter-server transfer time per unit of data
ℓ_s^c	Cloud service retrieval time per data unit
$X^{(k)}$	Cache configuration in period k
$x_{s,q}^{(k)}$	Binary server–service caching indicator
$a_{m,i,s}$	Binary task assignment variable
$s_{m,i}$	Task execution server index
$\Omega_{m,t}$	Tasks assigned after decision t
$F_{m,i}$	Task completion time from application arrival
T_m	Application completion time from arrival
$z_{m,i}$	Current-task offloading state
g_m	Application entry server index

pendency-respecting topological order; the entry server only selects their execution servers. The assigned-task set after decision t is $\Omega_{m,t}$, with $\Omega_{m,0} = \emptyset$, $\Omega_{m,N_m} = V_m$, and $N_m = |V_m|$.

Times are relative to application arrival. Task assignment steps specify decision order, while completion times follow (5). Table I lists the main notation.

B. Task Delay and Application Completion Time

Application m has entry server $g_m \in \{1, \dots, S\}$ and user-to-entry uplink rate R_{m,g_m}^u . Let $\ell_{s',s}$ denote transfer time per data unit over a direct link or predefined multihop path, with $\ell_{s,s} = 0$. Uploading initial task input and forwarding it to the execution server gives

$$\tau_{m,i,s}^{usr} = d_{m,i}^{\text{in}} \left(\frac{1}{R_{m,g_m}^u} + \ell_{g_m,s} \right). \quad (1)$$

With predecessor j completing at $F_{m,j}$ on server $s_{m,j}$, all immediate-predecessor data for task i reach server s at

$$R_{m,i,s}^{\text{pre}} = \begin{cases} 0, & \text{Pred}_{m,i} = \emptyset, \\ \max_{j \in \text{Pred}_{m,i}} (F_{m,j} + d_{m,j,i} \ell_{s_{m,j},s}), & \text{Pred}_{m,i} \neq \emptyset. \end{cases} \quad (2)$$

Let L_q be service q 's image size and ℓ_s^c server s 's cloud retrieval time per data unit. On a cache miss, select the lowest-delay source among the cloud and edge servers caching the service:

$$\phi_{s,q}(X^{(k)}) = \min \left\{ \ell_s^c, \min_{s': x_{s',q}^{(k)} = 1} \ell_{s',s} \right\}. \quad (3a)$$

$$\tau_{m,i,s}^{\text{suc}} = L_{q_{m,i}} \phi_{s,q_{m,i}}(X^{(k)}) (1 - x_{s,q_{m,i}}^{(k)}). \quad (3b)$$

Local hits incur zero loading delay. Without edge replicas, the inner minimum in (3a) is $+\infty$, selecting the cloud. All reachable servers, including those with $K_s = 0$, allow on-demand loading. Each miss incurs loading delay without changing $X^{(k)}$ or enabling reuse by later tasks.

Exogenous workload W_s , measured in CPU cycles, models background waiting, without explicitly modeling queuing contention among concurrent user tasks. Computation and background waiting delays are

$$\tau_{m,i,s}^{\text{cmp}} = \frac{c_{m,i}}{f_s}, \quad \tau_s^{\text{wait}} = \frac{W_s}{f_s}. \quad (4)$$

After all predecessor data arrive, initial input transfer, service loading, background waiting, and computation proceed sequentially, giving

$$F_{m,i} = R_{m,i,s}^{\text{pre}} + \tau_{m,i,s}^{\text{usr}} + \tau_{m,i,s}^{\text{suc}} + \tau_s^{\text{wait}} + \tau_{m,i,s}^{\text{cmp}}, \quad s = s_{m,i}. \quad (5)$$

The last completed exit task determines application completion: $T_m = \max_{i \in V_m^{\text{out}}} F_{m,i}$.

C. Joint Optimization Problem

π_C selects a cache configuration per period, and π_O selects a server for the current task.

History Γ_{k-1} contains only service requests, execution feedback, and cache configurations observed before period k . Given cache capacities K , image sizes L , and edge/cloud transfer parameters ℓ , caching follows $X^{(k)} = \pi_C(\Gamma_{k-1}, K, L, \ell)$.

State $z_{m,i}$ belongs to $\mathcal{Z}^{\text{cau}}$, using known static DAG attributes and decision-time information; Section III-B1 specifies its components. The offloading policy assigns tasks by

$$s_{m,i} = \pi_O(z_{m,i}), \quad z_{m,i} \in \mathcal{Z}^{\text{cau}}, \quad a_{m,i,s} = \mathbb{I}\{s = s_{m,i}\}, \quad s = 1, \dots, S. \quad (6)$$

$\mathbb{I}\{\cdot\}$ is the indicator function. Over M application instances, the objective is expected mean application completion time:

$$\begin{aligned} \text{P1:} \quad & \min_{\pi_C, \pi_O} \mathbb{E} \left[\frac{1}{M} \sum_{m=1}^M T_m \right] \\ \text{s.t.} \quad & \text{C1: } \sum_{s=1}^S a_{m,i,s} = 1, \quad \forall m, i \in V_m, \\ & \text{C2: } \sum_{q=1}^Q x_{s,q}^{(k)} \leq K_s, \quad \forall s, k, \\ & \text{C3: } a_{m,i,s}, x_{s,q}^{(k)} \in \{0, 1\}. \end{aligned} \quad (7)$$

C1 assigns each task to exactly one server; C2 enforces each server's cache capacity; C3 specifies the binary decision variables.

Caching affects service acquisition costs, while offloading changes server-level service demand, coupling the decisions.

III. TCOP ALGORITHM DESIGN

In Fig. 2, DCC coordinates caches using historical demand and execution feedback, distributing next-period configurations and execution-quality weights. Within each period, entry servers use Pairwise PD3QN for per-task server selection.

A. DCC Service Caching Coordination

1) *Historical Demand and Service Loading-Delay Savings:* DCC uses (3) to measure loading-delay savings relative to cloud-only loading. Let $\lambda_{s,q}^{(k)}$ be the exponential moving average (EMA) of the fraction of network-wide requests assigned to server s for service q . Configuration X yields demand-weighted savings

$$J_k(X) = \sum_{s=1}^S \sum_{q=1}^Q \lambda_{s,q}^{(k)} L_q [\ell_s^c - \phi_{s,q}(X)]. \quad (8)$$

Here, $\phi_{s,q}(X)$ follows (3a), and $J_k(X)$ is the proxy metric for service caching decisions. For period k , let n_k count all assigned tasks and $Y_{s,q}^{(k)}$ count server-service requests. DCC updates the demand weights by

$$\lambda_{s,q}^{(k)} = \begin{cases} (1 - \alpha_d) \lambda_{s,q}^{(k-1)} + \alpha_d \frac{Y_{s,q}^{(k)}}{n_k}, & n_k > 0, \\ \lambda_{s,q}^{(k-1)}, & n_k = 0. \end{cases} \quad (9)$$

Here, α_d is the smoothing coefficient. These statistics, execution feedback, and the current cache configuration form Γ_k .

2) *Execution Feedback and Cache Location Scoring:* For server s 's $n_{s,k} > 0$ assigned tasks in period k , let $\tau_{s,k}^{\text{obs}}$ denote the mean combined computation and background waiting delay, excluding service loading and predecessor communication. With smoothing coefficient α_h , the execution-delay EMA is

$$\mu_{s,k} = (1 - \alpha_h) \mu_{s,k-1} + \alpha_h \tau_{s,k}^{\text{obs}}, \quad n_{s,k} > 0. \quad (10)$$

Initialize $\mu_{s,k}$ at the first observation and retain it without new ones. Scoring uses $\mu_{s,k}^{\text{fb}}$: the local EMA if available, otherwise the EMA of network-wide mean task execution delay.

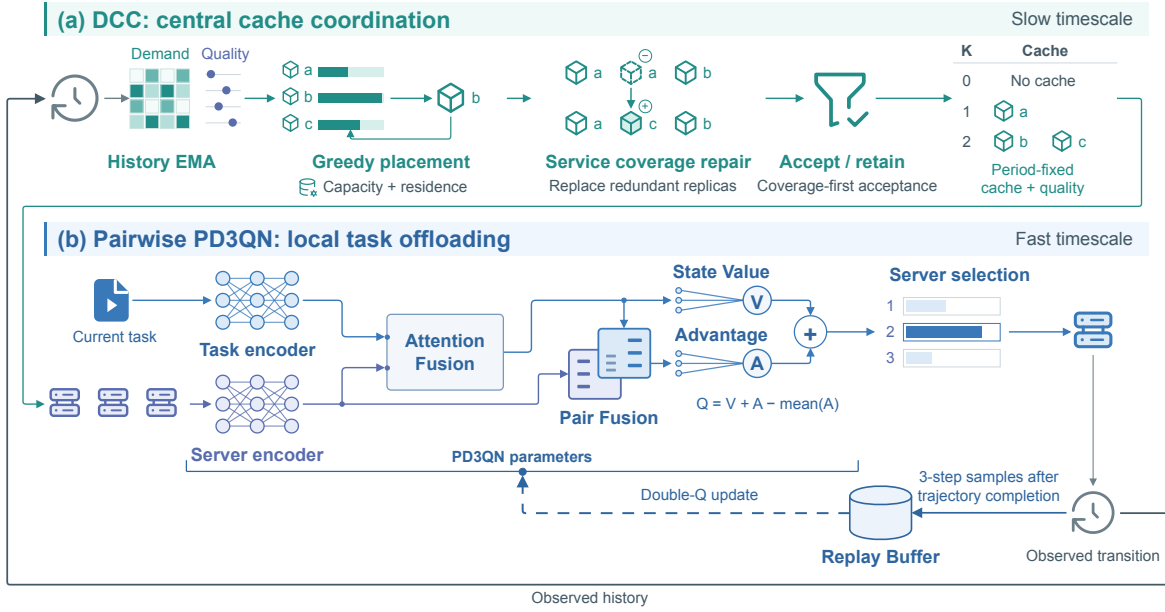


Fig. 2. Two-timescale framework of TCOP: (a) DCC service caching coordination; (b) Pairwise PD3QN offloading policy.

The same smoothing rule gives c_k^{ema} , the EMA of mean task CPU cycles, retained without requests. Let χ_k be $c_k^{ema}/c_{\max}$ clipped to $[0, 1]$, with common reference upper bound $c_{\max}$. Coefficient w_c controls the execution-quality adjustment:

$$\omega_{s,k} = \left(\frac{\min_{s'=1,\dots,S} \mu_{s',k}^{fb}}{\max \{ \mu_{s,k}^{fb}, \varepsilon_{num} \}} \right)^{w_c \chi_k}. \quad (11)$$

Here, ε_{num} prevents division by zero. Shorter delays yield higher weights; increasing χ_k further downweights slow servers. Weights start at 1 and equal 1 when $\chi_k = 0$. Since $\omega_k = (\omega_{1,k}, \dots, \omega_{S,k})$ takes effect next period, period- k decisions use ω_{k-1} .

Adding service q at server s to X gives X' , with marginal savings $\Delta_{s,q}^{(k)}(X) = J_k(X') - J_k(X)$.

With $\nu_q(X)$ denoting the replica count of service q , the addition score combines marginal savings, execution quality, and replica count:

$$\psi_{s,q}^{(k)}(X) = \frac{\omega_{s,k} \Delta_{s,q}^{(k)}(X)}{1 + \nu_q(X)}. \quad (12)$$

3) *Cache Configuration Construction and Update Decisions*: Only caches retained for at least $H_{\min}$ periods, as tracked by $h_s^{(k)}$, are eligible for change. DCC clears eligible rows in a candidate copy, then greedily adds feasible pairs with positive scores in (12).

On an eligible server s , coverage repair replaces cached service r with a service q having historical demand but no edge replica, producing X' . If r still has historical demand, another replica of r must remain. The replacement score is $g_{s,r,q}^{(k)}(X) = \omega_{s,k} [J_k(X') - J_k(X)]$.

Algorithm 1 obtains $X_c^{(k)}$ through feasible coverage-increasing replacements, regardless of score sign. Estimated reconfig-

uration cost sums the loading delays of newly cached services under the pre-update configuration:

$$C_{upd}(X^{(k)}, X_c^{(k)}) = \sum_{s=1}^S \sum_{\substack{q: x_{s,q}^{(k)}=0, \\ x_{c,s,q}^{(k)}=1}} L_q \phi_{s,q}(X^{(k)}). \quad (13a)$$

Let $n_k^{est} = \max\{n_k^{ema}, 1\}$, where n_k^{ema} is the EMA of requests per period, retained without requests. $Cov_k(X)$ counts services with positive historical demand on any server that are cached at least once. Define $\Delta Cov_k = Cov_k(X_c^{(k)}) - Cov_k(X^{(k)})$ and $\Delta J_k = J_k(X_c^{(k)}) - J_k(X^{(k)})$, abbreviating (13a) as C_{upd} . The update rule is

$$X^{(k+1)} = \begin{cases} X_c^{(k)}, & \Delta Cov_k > 0, \\ X_c^{(k)}, & \Delta Cov_k = 0 \text{ and } n_k^{est} \Delta J_k > \eta_{hys} C_{upd}, \\ X^{(k)}, & \text{otherwise.} \end{cases} \quad (13b)$$

After updating, set $h_s^{(k+1)} \leftarrow 0$ for changed caches and $h_s^{(k+1)} \leftarrow \min\{h_s^{(k)} + 1, H_{\min}\}$ otherwise. Clear period request counts, retaining demand and execution-delay EMAs and $h_s^{(k+1)}$.

B. Pairwise PD3QN Task Offloading

1) *State, Action, and Application-level Reward*: For application m and its current task i , the offloading state is

$$z_{m,i} = [\bar{F}_{m,i}^{pre}, \bar{c}_{m,i}, \bar{d}_{m,i}^{in}, \text{vec}(X^{(k)})], \quad (14)$$

$$p_{m,i}, \omega_{k-1}, e_{m,i}^{cur}, e_{m,i}^{suc}, \rho_{m,i}].$$

vec flattens the cache matrix; $p_{m,i}$ records immediate-predecessor locations; ω_{k-1} holds available historical execution-

Algorithm 1 DCC Service Caching Coordination

Input: $\Gamma_k, X^{(k)}, K, L, \ell$; retained EMAs and $h_s^{(k)}$.

Output: $X^{(k+1)}, \omega_k$.

- 1 Update EMAs and ω_k using (9)–(11).
 - 2 $X_g \leftarrow X^{(k)}$; clear entries with $h_s^{(k)} \geq H_{\min}$.
 - 3 **while** a feasible positive-score addition exists:
 - 4 Add (s, q) maximizing (12); update candidates.
 - 5 $X_c^{(k)} \leftarrow X_g$.
 - 6 **while** a feasible coverage-increasing replacement exists:
 - 7 Apply (s, r, q) maximizing $g_{s,r,q}^{(k)}(X)$.
 - 8 Obtain $X^{(k+1)}$ using (13a)–(13b).
 - 9 Update $h_s^{(k+1)}$; **return** $X^{(k+1)}, \omega_k$.
-

quality weights. Binary $e_{m,i}^{cur}$ and $e_{m,i}^{suc}$ indicate services required by the current task and its immediate successors. $\rho_{m,i}$ is task depth normalized by the application maximum: exits have depth 1; other depths equal one plus the maximum immediate-successor depth. Time is scaled as

$$\bar{F}_{m,i}^{pre} = \begin{cases} \frac{\max_{j \in \text{Pred}_{m,i}} F_{m,j}}{T_m^{ref}}, & \text{Pred}_{m,i} \neq \emptyset, \\ 0, & \text{Pred}_{m,i} = \emptyset. \end{cases} \quad (15)$$

Each user's time scale T_m^{ref} starts at 1 s and remains fixed within an application. During training, after all its real tasks are assigned, $T_m^{ref} \leftarrow 0.9T_m^{ref} + 0.1T_m$ tracks duration changes and smooths fluctuations for that user's next application. Workload and input size are scaled as

$$\bar{c}_{m,i} = \frac{c_{m,i}}{c_{\max}}, \quad \bar{d}_{m,i}^{in} = \frac{d_{m,i}^{in}}{D_{\max}}. \quad (16)$$

$c_{\max}$ is the CPU-cycle reference bound, and $D_{\max}$ the input-size reference (bits); all three scaled features remain unclipped.

The action $s_{m,i} \in \{1, \dots, S\}$ selects any reachable server; a cache miss incurs the loading cost in (3). The latest assigned-task completion time and incremental reward are

$$\Phi_{m,0} = 0, \quad \Phi_{m,t} = \max_{i \in \Omega_{m,t}} F_{m,i}, \quad t = 1, \dots, N_m. \quad (17a)$$

$$r_{m,t} = -\frac{\Phi_{m,t} - \Phi_{m,t-1}}{\zeta}, \quad \zeta > 0. \quad (17b)$$

After all tasks are assigned, $\Phi_{m,N_m} = T_m$; with $\gamma = 1$, the total return is

$$\sum_{t=1}^{N_m} r_{m,t} = -\frac{T_m}{\zeta}. \quad (18)$$

2) *Pairwise Encoding and Action Value Estimation:* Task and candidate-server features are encoded separately:

$$\begin{aligned} e_{m,i}^T &= f_T \left(\left[\bar{F}_{m,i}^{pre}, \bar{c}_{m,i}, \bar{d}_{m,i}^{in}, e_{m,i}^{cur}, e_{m,i}^{suc}, \rho_{m,i} \right] \right), \\ e_{m,i,s}^S &= f_S \left(\left[x_s^{(k)}, p_{m,i,s}, \omega_{s,k-1}, x_{s,q_{m,i}}^{(k)} \right] \right). \end{aligned} \quad (19)$$

Here, $x_s^{(k)}$ is row s of $X^{(k)}$. Encoders f_T, f_S have separate parameters and output d_h features using two fully connected layers with intervening layer normalization and activation; f_S is shared across servers. $p_{m,i,s}$ indicates immediate-predecessor presence at server s .

Scaled dot-product attention aggregates server embeddings using the task query [21]. Omitting m , the trainable projections are W_Q, W_K, W_V :

$$\alpha_{i,s} = \text{softmax}_s \left(\frac{\langle W_Q e_i^T, W_K e_{i,s}^S \rangle}{\sqrt{d_h}} \right). \quad (20a)$$

$$o_i = \sum_{s=1}^S \alpha_{i,s} W_V e_{i,s}^S. \quad (20b)$$

f_C maps concatenated e_i^T and o_i through a fully connected layer, normalization, and activation: $h_i = f_C([e_i^T; o_i])$.

The value branch uses h_i ; the advantage branch adds it to $e_{i,s}^S$, then normalizes and activates the result to form $h_{i,s}^P$ and score each server. Dueling aggregation gives [19]

$$Q_\theta(z_{m,i}, s) = V_\theta(h_i) + A_\theta(h_{i,s}^P) - \frac{1}{S} \sum_{s'=1}^S A_\theta(h_{i,s'}^P). \quad (21)$$

With online parameters θ , evaluation selects the server with the highest action value: $s_{m,i}^* = \arg \max_{s \in \{1, \dots, S\}} Q_\theta(z_{m,i}, s)$.

During training, if servers caching the current service exist, select one uniformly with probability $\beta = 0.1$. If this selection is not made, including when no server caches the service, apply ε -greedy selection over all candidates: select uniformly with probability ε , or choose the server with the highest action value with probability $1 - \varepsilon$. Evaluation sets $\beta = \varepsilon = 0$.

3) *Training and Two-timescale Interaction:* With z_t, s_t, r_t denoting pre-assignment state, action, and reward, construct 3-step samples from each fully assigned application's trajectory, truncating at its end. For $\gamma = 1$, the Double-Q target is [18]

$$y_t = \sum_{j=0}^{n_t-1} r_{t+j} + (1 - \delta_{t+n_t}) Q_{\theta^-}(z_{t+n_t}, s_{t+n_t}^*). \quad (22)$$

Here, $n_t = \min\{3, N_m - t + 1\}$, z_{t+n_t} is the resulting state, and $\delta_{t+n_t} = 1$ when the application's real tasks are all assigned (0 otherwise). Nonterminal targets use online selection of $s_{t+n_t}^*$ and evaluation by θ^- .

The online network minimizes the mean Huber loss between its action-value estimates and the targets in (22), using minibatches uniformly sampled from the entry server's replay buffer.

Periodic Adam updates [22] with gradient clipping and target-network synchronization follow Algorithm 2.

Each entry server shares a policy and replay buffer among its applications; $\pi_\theta^{(g_m)}$ implements π_O in Section II. During frozen-policy evaluation, trained parameters and caches are fixed, and exploration and training are disabled.

C. Computational Complexity of the Algorithm

With cache locations grouped by service, evaluating loading-delay savings costs $C_J = O(S(Q+B))$. DCC costs $O(BQ[S + \min\{B, Q\}]C_J)$ per cache update. Pairwise PD3QN has forward-pass cost $O(Sd_h(Q + d_h))$, linear in S for fixed Q, d_h .

Algorithm 2 Two-timescale Training of TCOP**Input:** Training DAGs $\{G_m\}$; initial cache $X^{(0)}$.**Output:** Per-entry-server policies $\pi_{\theta}^{(g_m)}$.

- 1 Initialize per-entry θ, θ^- and buffers, DCC state, and per-user $T_m^{ref} \leftarrow 1$ s.
- 2 **for each** training episode: reset trajectories and set $t = 0$ separately for each application.
- 3 **while** unassigned tasks remain:
- 4 Get task (m, i) from the environment; set $t \leftarrow t + 1$ for application m ; form z_t by (14).
- 5 Select s_t using training exploration; assign the task.
- 6 Compute r_t by (17b); record the transition and DCC statistics.
- 7 **if** $\Omega_{m,t} = V_m$:
- 8 Store its 3-step samples in the buffer at g_m ; update T_m^{ref} .
- 9 Complete the episode; advance the episode count.
- 10 **if** cache adaptation is active and a DCC update is due:
- 11 Run Algorithm 1; apply $X^{(k+1)}, \omega_k$ next episode.
- 12 **if** target synchronization is due: $\theta^- \leftarrow \theta$.
- 13 **if** a gradient update is due and replay samples suffice: update θ using Huber loss with targets from (22).
- 14 **return** per-entry-server policies $\pi_{\theta}^{(g_m)}$.

IV. EXPERIMENTAL EVALUATION

A. Experimental Setup

1) *Workflows and Simulation Environment:* Table II gives the environment and cache capacities. For each seed, all methods shared task instances, infrastructure, and a cache-capacity assignment randomly permuted across servers independently of location, computing capability, and background workload. Using Pegasus and WorkflowSim, we constructed five DAG types: CyberShake, Epigenomics, Inspiral, Montage, and Sipht [2], [20]. Each contained 24–30 real tasks and a zero-overhead virtual source; program–service correspondences were fixed across experiments. Completion time follows Section II-B, excluding result download. Network scale, area, background workloads, CPU frequencies, service sizes, communication parameters, and upper bounds on task data and computation followed DAOC’s public implementation [5]. Workloads and CPU frequencies were discrete-uniform, service sizes continuous-uniform, and cloud retrieval times equiprobable.

2) *Comparison Methods and Evaluation Protocol:* Random and Nearest use independent popularity-based caching and select a random server or the one nearest to the entry server, respectively; SA-Nearest prioritizes the nearest server caching the required service. DAOC-Cache independently updates each server’s cache from local demand and loading costs [5]; DCC coordinates caches centrally. DAOC and DAOC + DCC use DAOC’s distributed DQN with DAOC-Cache and DCC, respectively; discrete soft actor-critic (SAC) [23] is similarly paired. TCOP pairs Pairwise PD3QN with DCC. DQN-DAOC-Cache uses a basic DQN, distinct from Flat DDQN (Section IV-D).

Unless stated otherwise, experiments used 10 independent random seeds. The main comparison used 100 paired scenarios per seed (20 per workflow), each with one DAG of that type

TABLE II
NETWORK AND SIMULATION PARAMETERS

Parameter	Value or Range
Number of users	20
Number of edge servers	10
Number of service types	10
Simulation area	1 km $\times$ 1 km
Inter-server data rate	Approximately 31.5–40 Gbit/s
Cloud service retrieval time per bit	0.5 or 1.0 ns/bit
Server background workload	0.2–99.8 Mcycles
Server CPU frequency	0.2–29.8 Gcycles/s
Service image size	[10, 1000] Mbit
Task data volume	0–1.5 Mbit
Task computational workload	10–200 Mcycles
Wireless bandwidth	15 kHz
Number of cache slots per server	[0, 0, 0, 0, 1, 1, 1, 1, 2, 2]
Total number of cache slots	8

TABLE III
MAIN HYPERPARAMETERS OF TCOP

Hyperparameter	Value
State dimension (main environment)	144
Embedding dimension	64
Discount factor	1.0
Learning rate	5×10^{-4} (fixed)
Batch size	64
Replay buffer capacity	10,000
Gradient update interval	5 training episodes
Target network synchronization interval	20 training episodes
Minimum exploration rate	0.01
DCC cache update interval	5 training episodes

per user (20 users). Scenario-level completion-time means and 95th percentiles (P95s) were averaged across scenarios, then summarized across seeds by means and 95% Student-t confidence intervals. Bars and line markers, error bars, and scatter points denote means, intervals, and individual seed results, respectively.

3) *Training and Implementation Details:* Learning-based methods used 26,000 training episodes from random initialization, with hyperparameters tuned on development seeds and then fixed (Table III). Each episode ends after processing one DAG per user (20 applications); each real-task server selection is one decision. Cache and network update intervals are measured in completed episodes. DCC checks for conditional cache updates every 5 episodes during the first 5,000, then freezes the configuration.

DCC uses $\alpha_d = \alpha_h = 0.1$, $w_c = 1$, $H_{\min} = 2$ caching periods, and $\eta_{hys} = 1$; $\zeta = 1$ with completion time measured in seconds.

DAOC used two 64-unit tanh layers, with an action-guidance coefficient of 0.9 initially, 0.1 minimum, and 0.995 decay; minimum exploration was 0.01 [5]. Discrete SAC used 3-step returns, an initial entropy coefficient of 0.02, target entropy $0.98 \ln |\mathcal{A}|$ for action count $|\mathcal{A}|$, and a soft-update coefficient of 0.005.

Experiments ran on an Apple M3 with 16 GB RAM using Python 3.12.8, PyTorch 2.7.0, NumPy 1.26.4, SciPy 1.15.1, and SimPy 4.1.2.

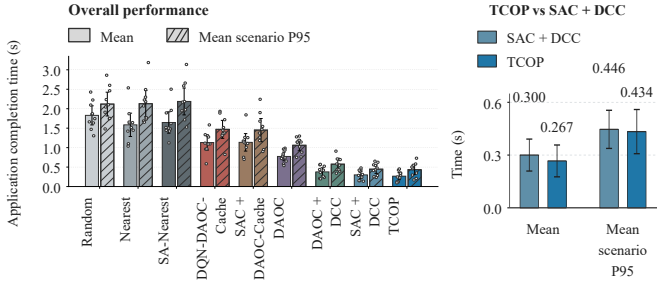


Fig. 3. Mean application completion time and mean scenario P95 for different methods in the main environment.

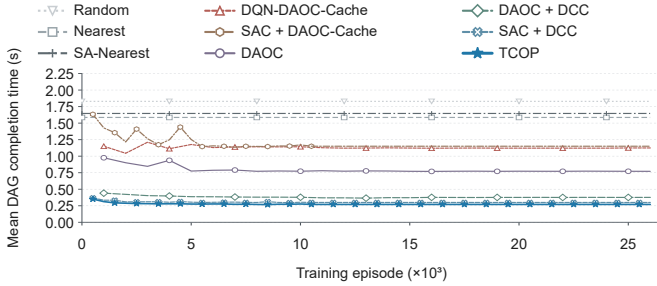


Fig. 4. Mean application completion time of different methods over training episodes.

B. Overall Performance and Training Evaluation

1) *Overall Performance in the Main Environment*: Fig. 3 shows mean application completion time and mean scenario P95. TCOP achieved 0.2668 s, 11.09% below SAC + DCC (0.3001 s), the lowest-mean baseline, and 65.51% below DAOC (0.7736 s); it was lower than both in each of the 10 seeds. Its mean scenario P95 was 0.4336 s, the lowest observed. In this comparison, heuristic methods had means of 1.5859–1.8290 s, all higher than those of learning-based methods, indicating that server distance or cache hits alone are insufficient for offloading.

2) *Validation Performance during Training*: In Fig. 4, TCOP’s mean application completion time fell rapidly to 0.2937 s by episode 1,500, below SAC + DCC’s 0.3002 s at episode 10,500. It fluctuated slightly around 0.27 s over episodes 5,000–10,500, consistently below SAC + DCC, and reached 0.2703 s at episode 10,500 (9.94% lower). These validation results show that TCOP learned a low-latency offloading policy within fewer episodes and retained its latency advantage after performance stabilized.

C. Mechanism Analysis

Service coverage is the fraction of service types cached at least once across edge servers. Hit rate and remote loading rate are the fractions of requests using a cached service at the execution server and retrieving one from another edge server or the cloud, respectively. In Fig. 5, DCC increased service coverage from 56% for DAOC and 46% for discrete SAC to 80%, the limit of 8 slots for 10 types. Cross-server coordi-

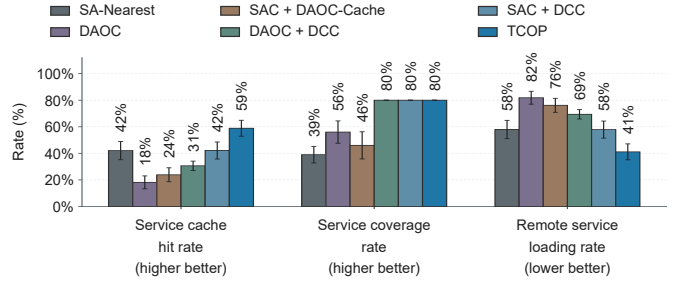


Fig. 5. Service cache hit rate, service coverage, and remote service loading rate for different methods.

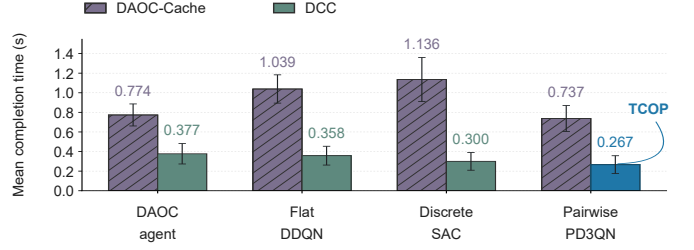


Fig. 6. Mean DAG application completion time of different offloading policies with DAOC-Cache and DCC.

nation reduced duplicate caching, leaving room for additional service types. At equal coverage, TCOP’s hit rate of 58.89% exceeded DAOC + DCC’s 30.62% and SAC + DCC’s 42.12%, indicating better cache utilization. Its remote loading rate was 41.11%; fewer remote loads and zero service loading delay for local hits (3b) help explain its lower application completion time in Fig. 3.

D. Ablation Study

Fig. 6 compares four offloading policies under DAOC-Cache and DCC: DAOC, discrete SAC, Flat DDQN, and Pairwise PD3QN. Flat DDQN and Pairwise PD3QN use the same state, reward, and return settings. DCC reduced mean application completion time by 51.23%–73.58% across the four policies. Pairwise PD3QN reduced it by 29.01% and 25.46% relative to Flat DDQN under DAOC-Cache and DCC, respectively. DCC with Pairwise PD3QN achieved the lowest mean, supporting the benefits of both modules in the tested combinations.

E. Workflow and Resource Configuration Sensitivity Experiments

1) *Performance across Workflows*: In Fig. 7, TCOP achieved the lowest mean application completion time across all five workflows. Reductions ranged from 0.89% (Epigenomics) to 22.34% (CyberShake) against SAC + DCC and 57.86%–71.90% against DAOC.

2) *Joint Scaling of Server Count and Cache Slots*: Fig. 8 compares performance as the number of edge servers increases from 5 to 20 and total cache slots from 4 to 16. Each setting was evaluated using three independent random seeds. With

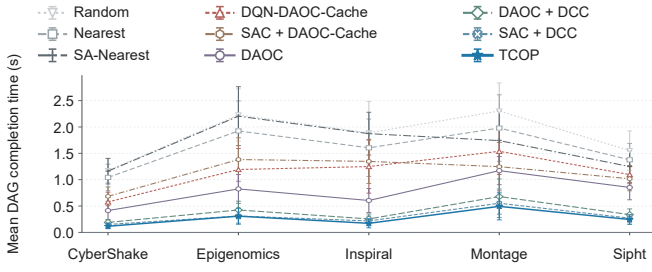


Fig. 7. Mean DAG application completion time for the five Pegasus workflows.

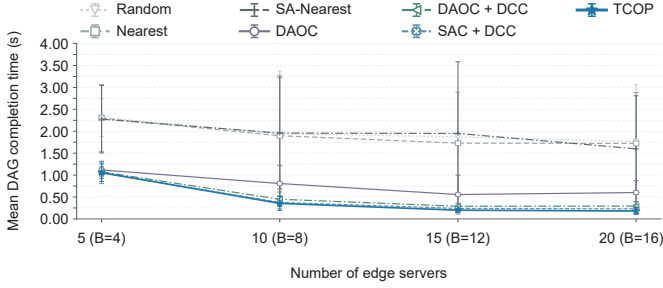


Fig. 8. Mean DAG application completion time as the number of servers and total cache slots change together.

joint resource scaling, TCOP's mean application completion time fell from 1.0549 to 0.1809 s, with a smaller additional reduction from 15 to 20 servers. Relative to SAC + DCC, it was 0.33% higher at 5 servers but 5.92%–21.25% lower at 10–20 servers, with an increasing advantage.

V. CONCLUSIONS AND FUTURE WORK

TCOP coordinates service caching and task offloading at two timescales in edge systems with heterogeneous cache capacities. Under the evaluated configurations, broader service coverage and improved cache utilization reduce mean application completion time. Future work could address dynamic arrivals, jointly model task queuing and service migration costs, and evaluate runtime overhead and application response time in real edge systems.

REFERENCES

- [1] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, 2017, doi: 10.1109/COMST.2017.2745201.
- [2] E. Deelman et al., "Pegasus, a workflow management system for science automation," *Future Gener. Comput. Syst.*, vol. 46, pp. 17–35, 2015, doi: 10.1016/j.future.2014.10.008.
- [3] G. Zhao, H. Xu, Y. Zhao, C. Qiao, and L. Huang, "Offloading tasks with dependency and service caching in mobile edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 11, pp. 2777–2792, 2021, doi: 10.1109/TPDS.2021.3076687.
- [4] Q. Shen, B.-J. Hu, and E. Xia, "Dependency-aware task offloading and service caching in vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 71, no. 12, pp. 13182–13197, 2022, doi: 10.1109/TVT.2022.3196544.
- [5] M. R. Golzari Oskoui and B. Sansò, "Distributed dependency-aware task offloading and service caching in cloudlet-based edge computing networks," *IEEE Trans. Services Comput.*, vol. 19, no. 2, pp. 1120–1133, 2026, doi: 10.1109/TSC.2026.3664339.
- [6] J. Xu, L. Chen, and P. Zhou, "Joint service caching and task offloading for mobile edge computing in dense networks," in *Proc. IEEE INFOCOM*, 2018, pp. 207–215, doi: 10.1109/INFOCOM.2018.8485977.
- [7] C. Sun, X. Li, C. Wang, Q. He, X. Wang, and V. C. M. Leung, "Hierarchical deep reinforcement learning for joint service caching and computation offloading in mobile edge-cloud computing," *IEEE Trans. Services Comput.*, vol. 17, no. 4, pp. 1548–1564, 2024, doi: 10.1109/TSC.2024.3355937.
- [8] G. Sun, M. Yuan, Z. Sun, J. Wang, H. Du, D. Niyato, Z. Han, and D. I. Kim, "Online collaborative resource allocation and task offloading for multi-access edge computing," *IEEE Trans. Mobile Comput.*, vol. 24, no. 11, pp. 11430–11448, 2025, doi: 10.1109/TMC.2025.3580365.
- [9] N. Yang, J. Wen, M. Zhang, and M. Tang, "Generalizable Pareto-optimal offloading with reinforcement learning in mobile edge computing," *IEEE Trans. Services Comput.*, vol. 18, no. 6, pp. 3824–3836, 2025, doi: 10.1109/TSC.2025.3604371.
- [10] C. Feng, P. Han, X. Zhang, Q. Zhang, Y. Liu, and L. Guo, "Dependency-aware task reconfiguration and offloading in multi-access edge cloud networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9271–9288, 2024, doi: 10.1109/TMC.2024.3360978.
- [11] Y. Zhou, L. Ma, Y. Qian, M. Huang, F. Hao, and X. Wang, "A stable locality-aware task scheduling mechanism for mobile edge computing with workflow task offloading," *IEEE Trans. Services Comput.*, vol. 19, no. 1, pp. 72–85, 2026, doi: 10.1109/TSC.2025.3640723.
- [12] Z. Wang, M. Goudarzi, and R. Buyya, "TF-DDRL: A Transformer-enhanced distributed DRL technique for scheduling IoT applications in edge and cloud computing environments," *IEEE Trans. Services Comput.*, vol. 18, no. 2, pp. 1039–1053, 2025, doi: 10.1109/TSC.2025.3528346.
- [13] Y. Liang, S. Zhang, and J. Wu, "Cost-efficient delay-bounded dependent task offloading with service caching at edges," *IEEE Trans. Comput.*, vol. 74, no. 11, pp. 3568–3581, 2025, doi: 10.1109/TC.2025.3598749.
- [14] G. Chen, W. Lin, F. Shi, H. Zhang, and B. Wang, "PAWSSP: A two-stage parallelism-aware algorithm for joint workflow scheduling and service placement in edge computing," *IEEE Trans. Services Comput.*, vol. 19, no. 1, pp. 572–588, 2026, doi: 10.1109/TSC.2025.3643326.
- [15] S. Yu, X. Chen, Z. Zhou, X. Gong, and D. Wu, "When deep reinforcement learning meets federated learning: Intelligent multitimescale resource management for multiaccess edge computing in 5G ultradense network," *IEEE Internet Things J.*, vol. 8, no. 4, pp. 2238–2251, 2021, doi: 10.1109/JIOT.2020.3026589.
- [16] L. Zhao, L. Sun, A. Hawbani, Z. Liu, X. Tang, and L. Xu, "Dual dependency-aware collaborative service caching and task offloading in vehicular edge computing," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 10963–10977, 2025, doi: 10.1109/TMC.2025.3573379.
- [17] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015, doi: 10.1038/nature14236.
- [18] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, pp. 2094–2100, 2016, doi: 10.1609/aaai.v30i1.10295.
- [19] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd International Conference on Machine Learning*, PMLR, vol. 48, pp. 1995–2003, 2016.
- [20] W. Chen and E. Deelman, "WorkflowSim: A toolkit for simulating scientific workflows in distributed environments," in *Proc. IEEE 8th International Conference on E-Science*, 2012, pp. 1–8, doi: 10.1109/eScience.2012.6404430.
- [21] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems 30*, pp. 5998–6008, 2017.
- [22] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations*, 2015.
- [23] P. Christodoulou, "Soft actor-critic for discrete action settings," arXiv preprint arXiv:1910.07207, 2019, doi: 10.48550/arXiv.1910.07207.